C#

Complete Language Notes

From Basics to Advanced

A structured study guide covering syntax, examples, object-oriented programming, modern C# features, .NET, testing, databases, security, and practice projects.

For: beginners, students, interview preparation, and developers who want a strong foundation in C#.

How to study: type and run the examples, change them, read compiler errors carefully, and complete the exercises.

Examples use modern C# syntax where noted. Some features require a newer C# language version or target framework.

Contents

1. C# and .NET overview
2. Setup and first program
3. Syntax, variables, and types
4. Operators and expressions
5. Strings and text
6. Decisions and loops
7. Arrays and collections
8. Methods and parameters
9. Object-oriented programming
10. Properties, records, structs, and enums
11. Interfaces and polymorphism
12. Exceptions and error handling
13. Generics
14. Delegates, lambdas, and events
15. LINQ
16. Files, streams, and serialization
17. Nullable reference types
18. Async and await
19. Tasks, threads, and parallelism
20. Memory management and IDisposable
21. Advanced language features
22. .NET project structure and packages
23. Testing, debugging, logging, and style
24. Databases and web APIs
25. Security and performance
26. Common mistakes
27. Practice projects and interview review
28. Quick reference and roadmap

1. C# and .NET overview

C# (pronounced “C sharp”) is a strongly typed, general-purpose language used for web services, desktop applications, cloud services, games, and enterprise software.

C# versus .NET

• **C#** is the language: syntax, types, operators, and language features.

• **.NET** is the platform: runtime, libraries, SDK, build tools, and application frameworks.

• **CLR** (Common Language Runtime) executes managed code and provides services such as garbage collection.

• **NuGet** is the package ecosystem for .NET libraries.

C# source is compiled to Intermediate Language (IL) and metadata. The .NET runtime typically uses just-in-time compilation; ahead-of-time options also exist.

2. Setup and first program

```
dotnet --info
dotnet new console -n HelloCSharp
cd HelloCSharp
dotnet run
```

First program

```
Console.WriteLine("Hello, world!");
Console.Write("Enter your name: ");
string? name = Console.ReadLine();
Console.WriteLine($"Hello, {name}!");
```

Modern console templates may use top-level statements. Traditional structure:

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello, world!");
    }
}
```

`Console.WriteLine` prints a line. `Console.ReadLine` reads a line and may return null.

3. Syntax, variables, and types

```
int age = 20;
string name = "Ravi";
// Single-line comment
if (age >= 18)
{
    Console.WriteLine("Adult");
}
```

C# is case-sensitive. Statements generally end with semicolons, and braces define blocks.

Type	Example	Purpose
bool	true	Boolean value
byte / sbyte	byte b = 255;	8-bit unsigned / signed integer
short / ushort	short n = -10;	16-bit integer types
int / uint	int count = 10;	32-bit integer types
long / ulong	long big = 9000000L;	64-bit integer types
float	float f = 1.2f;	32-bit floating point
double	double d = 3.14;	64-bit floating point
decimal	decimal price = 19.99m;	Decimal arithmetic, often money
char	char c = 'A';	UTF-16 code unit
string	string text = "Hi";	Immutable text

Type	Example	Purpose
object	object value = 42;	Base type for all .NET types

Value types and reference types

Value types include numeric types, bool, char, enums, and structs. Classes, arrays, strings, and delegates are reference types. Value-type assignment copies the value; reference-type assignment copies a reference to the same object.

var, constants, and nullable values

```
var city = "Warangal";        // inferred as string at compile time
const double Pi = 3.14159;
int? optionalNumber = null;
string? optionalName = null;
```

`var` is compile-time type inference, not dynamic typing. `const` is a compile-time constant.

Conversions and parsing

```
int x = 10;
double y = x;               // implicit widening conversion
int z = (int)3.9;            // explicit cast gives 3
if (int.TryParse("123", out int number))
    Console.WriteLine(number);
```

Use TryParse when invalid input is expected. Culture-sensitive parsing may need an explicit culture.

4. Operators and expressions

Category	Operators	Notes
Arithmetic	+ - * / %	Integer division truncates toward zero
Comparison	== != < > <= >=	Compare values
Logical	&& || !	Short-circuit boolean operators
Bitwise	& | ^ ~ << >>	Bit-level operations
Assignment	= += -= *= ??=	Assign or update
Null	?? ?. ??=	Fallback, conditional access, null assignment
Conditional	condition ? a : b	Choose an expression
Type	is as typeof	Test or inspect types

```
Console.WriteLine(7 / 2);       // 3
Console.WriteLine(7.0 / 2);     // 3.5
int? maybe = null;
int actual = maybe ?? 0;
string? input = null;
int length = input?.Length ?? 0;
```

Use parentheses for clarity. Be deliberate about integer division, overflow, and precedence.

Checked arithmetic

```
checked
{
    int n = int.MaxValue;
    // int next = n + 1; // throws OverflowException in checked context
}
```

5. Strings and text

```
string language = "C#";
string greeting = $"Hello, {language}";
string[] parts = "red,green,blue".Split(',');
string cleaned = "  hello  ".Trim();
bool starts = greeting.StartsWith("Hello", StringComparison.Ordinal);
```

Strings are immutable; operations return new strings. For repeated concatenation in a large loop, consider StringBuilder.

```
using System.Text;
var builder = new StringBuilder();
builder.Append("C#");
builder.Append(" notes");
```

```
string result = builder.ToString();
```

String comparison

Choose comparison rules deliberately. Use ordinal comparison for machine identifiers and culture-aware comparison for human-language sorting. Do not assume all cultures sort or case-fold text identically.

Verbatim and raw strings

```
string path = @"C:\Users\Public\notes.txt";
string json = """
{
  "language": "C#",
  "level": "advanced"
}
"""; // raw string literal, C# 11+
```

6. Decisions and loops

```
int score = 82;
if (score >= 90)
    Console.WriteLine("A");
else if (score >= 75)
    Console.WriteLine("B");
else
    Console.WriteLine("Keep practicing");

switch (score)
{
    case >= 90:
        Console.WriteLine("Excellent");
        break;
    case >= 75:
        Console.WriteLine("Good");
        break;
    default:
        Console.WriteLine("Review");
        break;
}
```

Loops

```
for (int i = 0; i < 3; i++)
    Console.WriteLine(i);

int n = 3;
while (n > 0)
{
    Console.WriteLine(n);
    n--;
}

foreach (string item in new[] { "A", "B" })
    Console.WriteLine(item);
```

- `break` exits the nearest loop or switch.
- `continue` skips to the next loop iteration.
- Use foreach when you need each item; use for when index control matters.

Switch expressions

```
string label = score switch
{
    >= 90 => "Excellent",
    >= 75 => "Good",
    >= 50 => "Pass",
    _ => "Try again"
};
```

7. Arrays and collections

```
int[] values = { 10, 20, 30 };
Console.WriteLine(values[0]);
values[1] = 25;
int[] sized = new int[5];

var names = new List<string> { "Asha", "Bo" };
names.Add("Chen");
names.Remove("Bo");

var ages = new Dictionary<string, int>();
ages["Asha"] = 22;
bool found = ages.TryGetValue("Asha", out int age);

var unique = new HashSet<int> { 1, 2, 2, 3 };
var queue = new Queue<string>();
queue.Enqueue("first");
string first = queue.Dequeue();

var stack = new Stack<int>();
stack.Push(10);
int top = stack.Pop();
```

Collection	Best for	Typical note
Array	Fixed-size indexed sequence	Fast indexed access
List<T>	Resizable ordered sequence	Common general-purpose list
Dictionary<TKey,TValue>	Key-value lookup	Keys are unique
HashSet<T>	Unique values and membership	Set operations
Queue<T>	First-in, first-out	Enqueue / Dequeue
Stack<T>	Last-in, first-out	Push / Pop

Useful abstractions include IEnumerable, IReadOnlyList, ICollection, and IDictionary. Accept the least specific interface that provides the operations you need.

8. Methods and parameters

```
static int Add(int a, int b)
{
    return a + b;
}

static void PrintName(string name = "Guest")
{
    Console.WriteLine(name);
}

int sum = Add(3, 4);
PrintName();
PrintName("Mina");
```

A method returns a value or void. Parameters can have defaults; callers can use named arguments.

ref, out, in, params

```
static bool TryGetAge(string text, out int age)
{
    return int.TryParse(text, out age);
}

static int Sum(params int[] numbers) => numbers.Sum();
```

`out` communicates an output value; `ref` passes an initialized variable by reference; `in` passes a readonly reference where supported; `params` accepts a variable number of arguments.

Overloading and expression-bodied methods

```
static int Area(int side) => side * side;
static double Area(double width, double height) => width * height;
static int Square(int n) => n * n;
```

Overloads share a name but differ in parameter signatures. Avoid overloads that make calls ambiguous.

9. Object-oriented programming

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }

    public string Describe() => $"{Name}, age {Age}";
}

var person = new Person("Asha", 25);
Console.WriteLine(person.Describe());
```

A class defines instance shape and behavior. Constructors initialize instances. Access modifiers include public, private, protected, internal, and combinations such as protected internal.

Inheritance and polymorphism

```
public class Animal
{
    public virtual string Speak() => "sound";
}
public class Dog : Animal
{
    public override string Speak() => "woof";
}
Animal animal = new Dog();
Console.WriteLine(animal.Speak()); // woof
```

virtual allows overriding; override supplies a replacement. Polymorphism lets code work through a base type while invoking the derived implementation.

Abstract and sealed

```
public abstract class Shape
{
    public abstract double Area();
}
public sealed class Circle : Shape
{
    public double Radius { get; }
    public Circle(double radius) => Radius = radius;
    public override double Area() => Math.PI * Radius * Radius;
}
```

Abstract classes cannot be instantiated directly. Sealed classes cannot be inherited from. Favor composition when an object uses another service rather than truly being a subtype.

10. Properties, records, structs, and enums

```
public class Product
{
    public string Name { get; private set; }
    public decimal Price { get; init; }
    public Product(string name) => Name = name;
}

public record Customer(int Id, string Name);
var c1 = new Customer(1, "Asha");
var c2 = c1 with { Name = "Mina" };

enum OrderStatus { Pending, Paid, Shipped, Cancelled }
```

Properties provide controlled access to state. An init-only property is assignable during initialization. Records are designed for data-centric types and provide value-based equality. A record class remains a reference type; a record struct is a value type.

Structs

```
public readonly struct Coordinate
{
    public double X { get; }
    public double Y { get; }
    public Coordinate(double x, double y) => (X, Y) = (x, y);
}
```

Structs are value types, often useful for small value-like data. Large mutable structs can be costly or confusing due to copying.

11. Interfaces and polymorphism

```
public interface INotifier
{
    void Send(string message);
}

public class EmailNotifier : INotifier
{
    public void Send(string message)
        => Console.WriteLine($"Email: {message}");
}

static void Notify(INotifier notifier)
{
    notifier.Send("Build complete");
}
```

An interface defines a contract. Classes and structs can implement multiple interfaces. Dependency injection supplies services from outside a class and improves testability. ASP.NET Core includes a built-in DI container.

12. Exceptions and error handling

```
try
{
    int number = int.Parse("abc");
}
catch (FormatException ex)
{
    Console.WriteLine($"Invalid number: {ex.Message}");
}
finally
{
    Console.WriteLine("Done");
}
```

Catch exceptions you can handle meaningfully. Avoid catching and ignoring errors. Use standard exception types when appropriate.

```
static decimal Divide(decimal a, decimal b)
{
    if (b == 0)
        throw new DivideByZeroException("Divisor cannot be zero.");
    return a / b;
}

public class InvalidOrderException : Exception
{
    public InvalidOrderException(string message) : base(message) { }
}
```

Use finally for cleanup when needed, but prefer using for disposable resources. Do not reveal sensitive internal details in errors shown to users.

13. Generics

```
public class Box<T>
{
    public T Value { get; set; }
    public Box(T value) => Value = value;
}
var intBox = new Box<int>(42);
var textBox = new Box<string>("hello");

static T First<T>(IReadOnlyList<T> items) => items[0];
static T Create<T>() where T : new() => new T();
```

Generics provide type safety and reusable code. Constraints such as class, struct, notnull, interface/base-class constraints, and new() restrict allowed type arguments.

Variance uses out (covariance) and in (contravariance) on suitable interfaces and delegates. For example, IEnumerable is covariant.

14. Delegates, lambdas, and events

```
Func<int, int> square = x => x * x;
Func<int, int, int> add = (a, b) => a + b;
Action<string> log = message => Console.WriteLine(message);

public class Alarm
{
    public event EventHandler? Ring;
    public void Trigger() => Ring?.Invoke(this, EventArgs.Empty);
}
```

Delegates are type-safe method references. Common delegate types are Action, Func, and Predicate. Events implement publish/subscribe. Unsubscribe when necessary so publishers do not retain subscribers longer than intended.

15. LINQ

```
int[] values = { 1, 2, 3, 4, 5, 6 };
var evenSquares = values
    .Where(n => n % 2 == 0)
    .Select(n => n * n)
    .ToList();

int total = values.Sum();
int largest = values.Max();
bool anyLarge = values.Any(n => n > 5);
```

Operator	Purpose
Where	Filter by predicate
Select	Project each item
OrderBy / ThenBy	Sort
GroupBy	Group by key
Join	Match items from sequences
Any / All	Test predicates
First / FirstOrDefault	Get first matching item
Single / SingleOrDefault	Expect at most one match
Count / Sum / Min / Max	Aggregate
ToList / ToArray	Materialize a query

Query syntax and deferred execution

```
var query =
    from n in values
    where n > 2
    orderby n descending
    select n;
```

Many LINQ queries are deferred until enumerated. Enumerating repeatedly may repeat work. ToList materializes a snapshot. First throws when no item matches; FirstOrDefault returns a default value, which may be ambiguous for value types.

16. Files, streams, and serialization

```
using System.IO;

string path = "notes.txt";
File.WriteAllText(path, "Hello C#");
string text = File.ReadAllText(path);

foreach (string line in File.ReadLines(path))
    Console.WriteLine(line);

using var stream = File.OpenRead("data.bin");
byte[] buffer = new byte[4096];
int bytesRead = stream.Read(buffer, 0, buffer.Length);
```

Use Path.Combine for portable path composition. Dispose streams with using. ReadLines can process lines lazily.

JSON

```
using System.Text.Json;

var data = new { Name = "Asha", Score = 95 };
string json = JsonSerializer.Serialize(data);
```

For deserialization, prefer known DTO types and validate data. CSV parsing is more complex than splitting on commas because fields may contain quotes, commas, and line breaks; use a suitable CSV library for general data.

17. Nullable reference types

```
#nullable enable
string name = "Asha";
string? optionalName = null;

int length = name.Length;
int safeLength = optionalName?.Length ?? 0;
```

Nullable reference types help the compiler identify possible null use. The ? annotation is mainly compile-time analysis; it does not validate external data at runtime.

```
string? maybe = GetValue();
Console.WriteLine(maybe!.Length); // only if you know it is non-null
```

The null-forgiving operator suppresses a warning but performs no runtime null check. Prefer validating or correcting the program's flow.

18. Async and await

```
static async Task<string> GetTextAsync(HttpClient client, string url)
{
    return await client.GetStringAsync(url);
}

using var client = new HttpClient();
string content = await GetTextAsync(client, "https://example.com");
```

Async programming helps apps stay responsive while waiting for I/O; it does not automatically speed up CPU-heavy work.

- Use async APIs end-to-end for I/O where available.
- Avoid async void except for event handlers.
- Prefer await over blocking on Task.Result or Task.Wait().
- Use CancellationToken to support cancellation.
- Use Task.WhenAll for independent operations that should run concurrently.

```
static async Task RunAsync(CancellationToken token)
{
    await Task.Delay(1000, token);
}
```

19. Tasks, threads, and parallelism

```
Task<int> a = Task.Run(() => 20);
Task<int> b = Task.Run(() => 22);
int[] results = await Task.WhenAll(a, b);
Console.WriteLine(results.Sum());
```

A thread is an OS execution unit. A Task represents an asynchronous operation or scheduled work; tasks do not map one-to-one to threads.

```
Parallel.For(0, 100, i =>
{
    // Independent CPU work
});
```

- Use async I/O for scalable waiting on network or file operations.
- Use Task.Run or parallel APIs for suitable CPU-bound work when parallelism helps.
- Shared mutable state requires synchronization.
- Parallel overhead can make small tasks slower; measure first.

20. Memory management and IDisposable

.NET garbage collection reclaims unreachable managed objects. It does not replace deterministic cleanup for file handles, sockets, and other external resources.

```
using var reader = new StreamReader("notes.txt");
string? line = reader.ReadLine();
```

Types that own disposable resources may implement IDisposable or IAsyncDisposable. Prefer SafeHandle and the dispose pattern for unmanaged resources rather than writing finalizers without a specific need.

Performance basics

• Measure before optimizing with a profiler or benchmark.

• Choose suitable algorithms and data structures.

• Stream large files instead of loading everything into memory.

• Understand allocations and copying when performance matters.

• Use caching only when correctness, invalidation, and memory costs are understood.

21. Advanced language features

Pattern matching

```
object value = 42;
string description = value switch
{
    int n when n > 0 => $"Positive integer: {n}",
    int => "Integer",
    string text => $"Text: {text}",
    null => "Null",
    _ => "Other"
};
```

Tuples and deconstruction

```
(string Name, int Age) person = ("Mina", 28);
var (name, age) = person;
```

Extension methods

```
public static class StringExtensions
{
    public static bool IsBlank(this string? value)
        => string.IsNullOrWhiteSpace(value);
}
// Example: " ".IsBlank()
```

Extension methods add callable syntax without changing the original type. Keep them discoverable and avoid overuse.

Attributes, reflection, dynamic

```
[Obsolete("Use NewMethod instead")]
static void OldMethod() { }

Type type = typeof(string);
Console.WriteLine(type.FullName);

dynamic item = "hello";
Console.WriteLine(item.Length);
```

Attributes attach metadata. Reflection inspects types at runtime but can reduce compile-time safety. dynamic defers certain member-binding checks until runtime.

Preprocessor and memory views

```
#if DEBUG
Console.WriteLine("Debug build");
#endif
```

Span and ReadOnlySpan provide efficient views over contiguous memory and can avoid allocations in performance-sensitive code, but have lifetime restrictions. Use them when profiling demonstrates a benefit.

22. .NET project structure and packages

```
dotnet new console -n MyApp
dotnet run
dotnet build
dotnet test
dotnet publish -c Release
```

SDK-style project file

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

This is illustrative; select a target framework supported by your SDK and deployment environment.

NuGet

```
dotnet add package PackageName
dotnet list package
dotnet restore
```

Review package maintenance, licenses, security advisories, and transitive dependencies. Use dependency update and lockfile practices appropriate to the project.

23. Testing, debugging, logging, and style

```
using Xunit;

public class CalculatorTests
{
    [Fact]
    public void Add_ReturnsSum()
    {
        Assert.Equal(5, Calculator.Add(2, 3));
    }
}
```

This example uses xUnit. NUnit and MSTest are other options. Test normal behavior, boundaries, invalid input, and failure paths. Unit tests should be repeatable and independent where possible.

Debugging checklist

- Read exception type, message, and stack trace.
- Set breakpoints and inspect values and call stacks.
- Build a minimal reproducible example.
- Check nulls, indexes, culture, file paths, and async exceptions.

Logging

```
using Microsoft.Extensions.Logging;

using ILoggerFactory factory = LoggerFactory.Create(builder =>
    builder.AddConsole());
ILogger logger = factory.CreateLogger("App");
logger.LogInformation("Application started");
```

Logging packages and providers depend on the project. Use structured logs and never log secrets, passwords, or access tokens.

Style

- Use PascalCase for public types and members; camelCase for local variables and parameters.
- Keep methods focused and names descriptive.
- Use analyzers and formatters where practical.
- Document public APIs and non-obvious decisions.
- Prefer explicit error handling over silent failure.

24. Databases and web APIs

SQL parameterization

```
using Microsoft.Data.Sqlite;

using var connection = new SqliteConnection("Data Source=app.db");
connection.Open();
using var command = connection.CreateCommand();
command.CommandText = "SELECT Name FROM Users WHERE Id = $id";
command.Parameters.AddWithValue("$id", 1);
using var reader = command.ExecuteReader();
while (reader.Read())
    Console.WriteLine(reader.GetString(0));
```

This example requires a compatible SQLite provider package. Parameterized queries help prevent SQL injection; do not concatenate untrusted values into SQL.

HTTP client

```
using var client = new HttpClient();
using HttpResponseMessage response =
    await client.GetAsync("https://example.com");
response.EnsureSuccessStatusCode();
string body = await response.Content.ReadAsStringAsync();
```

For long-running server applications, use a managed or injected HttpClient pattern such as IHttpClientFactory rather than creating a new client for every request. Set timeouts and cancellation behavior.

ASP.NET Core overview

• Routing maps URLs and HTTP methods to endpoint handlers.

• Dependency injection supplies services.

• Middleware forms a pipeline for logging, authentication, and error handling.

• Model binding and validation handle request data.

• Authentication verifies identity; authorization checks permissions.

25. Security and performance

• Validate untrusted input at application boundaries.

• Use parameterized database commands.

• Store secrets outside source control and restrict access.

• Use HTTPS and validate certificates.

• Keep dependencies updated and review advisories.

• Avoid unsafe deserialization of untrusted data.

• Enforce authorization on the server, not only in the UI.

• Do not expose stack traces or secrets in public error responses.

• Benchmark release builds with realistic data.

Operation	Typical complexity
Array/list indexed access	O(1)
Linear search in a list	O(n)
HashSet/Dictionary lookup	Average O(1)
Comparison sorting	O(n log n)
Loop through n elements	O(n)

26. Common mistakes

Mistake	Better approach
Assuming integer division returns a fraction	Use double or decimal operands deliberately
Ignoring nullable warnings	Validate nulls and enable nullable analysis
Catching and ignoring exceptions	Handle meaningfully or propagate
Forgetting to dispose streams	Use using or await using
Creating HttpClient for every request in a server	Use managed/injected client patterns
Concatenating strings repeatedly in hot loops	Consider StringBuilder if profiling supports it
Modifying a collection during enumeration	Use a safe update strategy
Blocking async code with .Result	Use await in async flows
Concatenating user input into SQL	Use parameters
Assuming async means parallel CPU execution	Choose the right execution model

27. Practice projects and interview review

Beginner projects

• Calculator: arithmetic, invalid input, and divide-by-zero handling.

• Number guessing game: random target, attempt counter, and hints.

• Student grade manager: scores, averages, and validation.

• Unit converter: temperature, distance, and weight.

Intermediate projects

• To-do list: add, complete, search, and save tasks to JSON.

• Contact book: classes, collections, validation, and persistence.

• Expense tracker: SQLite storage and category reports.

• Log analyzer: parse lines, group severity, and summarize errors.

Advanced projects

• REST API with ASP.NET Core, validation, dependency injection, logging, and tests.

• Async downloader with cancellation, timeouts, bounded concurrency, and retries.

• Reusable library with generics, nullable annotations, XML docs, packaging, and tests.

• Background worker with queues, graceful shutdown, and telemetry.

Interview questions

• What is the difference between C# and .NET?

• Explain value types versus reference types.

• What are boxing and unboxing?

• Compare an interface with an abstract class.

• Explain overloading and overriding.

• What are delegates and events?

• What is LINQ deferred execution?

• What do async and await do?

• Why use IDisposable?

• What is dependency injection?

• What do nullable reference types provide?

• How do parameterized SQL queries improve security?

Coding practice

```
static bool IsEven(int n) => n % 2 == 0;

static int Factorial(int n)
{
    if (n < 0) throw new ArgumentOutOfRangeException(nameof(n));
    int result = 1;
    for (int i = 2; i <= n; i++)
        result = checked(result * i);
    return result;
}

// More practice:
// - Count word frequency in a string.
// - Return the second-largest distinct integer.
// - Group objects by a property using LINQ.
// - Parse and validate CSV rows with a proper library.
// - Write tests for empty and boundary inputs.
```

28. Quick reference and roadmap

```
int count = 10;
string? optional = null;
var items = new List<int> { 1, 2, 3 };

if (count > 0) { Console.WriteLine(count); }
for (int i = 0; i < items.Count; i++) { }
foreach (int item in items) { }

static int Add(int a, int b) => a + b;

try { /* work */ }
catch (FormatException ex) { Console.WriteLine(ex.Message); }

var evens = items.Where(x => x % 2 == 0).ToList();

using var reader = new StreamReader("file.txt");
```

Stage	Focus	Checkpoint
1. Foundations	Types, operators, strings, decisions, loops	Write console programs independently
2. Core language	Collections, methods, classes, exceptions, files	Build a tested CLI app
3. Modern C#	Generics, LINQ, records, nullable types, delegates	Create a reusable library
4. .NET workflow	SDK, packages, tests, logging, debugging, Git	Build from a clean checkout
5. Specialization	ASP.NET Core, desktop, games, data, cloud	Complete a portfolio project
6. Advanced	Async, performance, memory, security, architecture	Profile and maintain a realistic app

Final checklist

• I can choose suitable types and collections.

• I can write methods and classes with clear responsibilities.

• I understand interfaces, inheritance, and composition.

• I can use exceptions, files, JSON, and LINQ safely.

• I understand async/await, cancellation, and disposal.

• I can write tests and interpret stack traces.

• I know basic security practices for SQL, HTTP, secrets, and external data.

Note: This guide is a broad learning reference. Exact syntax, templates, and APIs vary by C# language version and target .NET framework. Consult official C# and .NET documentation for version-specific details.